# Dive into Deep Learning (D2L)

## Top-Down Approach—LighTellite: Reinforcement Learning-Based Framework for Energy Efficient Onboard Satellite Anomaly Detection

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor
Defense & Advanced Security (DAS) Lab
Jeonbuk National University (JBNU)

# 01. Abstract

# 1. Motivation: 큐브위성의 에너지-보안 트레이드오프

- **이상 탐지(AD)의 필수성**: 위성의 안정적인 운영을 위해서는 온보드 시스템의 비정상 동작을 실시간으로 감시하는 기술이 필수적임
- **자원 제약의 장벽**: 큐브위성(CubeSat)은 에너지, 메모리, 연산 자원이 극도로 제한된 환경임
- **전통적 방식의 한계**: 기존 AD 모델은 높은 연산량을 요구하여 큐브위성에 적용 시 성능 저하나 배터리 고갈 문제를 야기함

**The Core Challenge**

에너지 소비를 최소화하면서도 보안성(탐지율)을 유지할 수 있는 지능형 온보드 시스템이 필요합니다.

# 2. Proposed Solution: LighTellite

- **강화학습 기반 Dual-Agent 구조**: 상황에 따라 에너지를 배분하고 모델을 선택하는 지능형 시스템
- **에이전트별 역할 분담**:
  1. **Manager Agent**: 위성의 상태를 보고 현재 구간에서 사용할 **에너지 예산**을 결정
  2. **Worker Agent**: 예산 범위 내에서 **최적의 이상 탐지 모델**을 동적으로 선택(DMS)
- **문맥 인식 적응(Context-aware)**: 온보드 데이터와 남은 자원을 실시간으로 분석하여 모델을 스위칭함

# 3. Performance Impact: AegisSat 실험 결과

- **실험 환경**: 최신 큐브위성 테스트베드인 **AegisSat**을 통해 실증 수행
- **정량적 우수성** (기존 Static 모델 대비):
  - **공격 탐지율(TPR)**: 약 **10% 향상**
  - **에너지 소비량**: 추론 과정에서 **21.8% 절감**

**Performance Comparison (LighTellite vs. Static)**

| 지표 (Metric) | Static AD Model | **LighTellite (Ours)** |
|---|---|---|
| Attack Detection Rate | Baseline (100%) | **+ 10% 개선** |
| Energy Consumption | Baseline (100%) | **- 21.8% 절감** |

**Summary**

LighTellite는 자원이 제한된 위성 플랫폼에서 "성능"과 "지속 가능성"을 동시에 확보할 수 있음을 증명했습니다.

# 02. Introduction

# 1. 왜 위성에서 이상 탐지인가?

- **역할**: 복잡한 시스템에서 라벨링되지 않은 예상치 못한 동작이나 악의적인 행위를 탐지함
- **원리**: 정상 운영 패턴을 학습하여, 이와 상충하는 실패(Failure)나 사이버 공격을 식별함
- **도메인 특성**: 위성, 특히 큐브위성(CubeSat)은 실패 시 막대한 손실이 발생하는 고신뢰성 시스템임

**Challenge**

큐브위성은 접근성(제작이 쉬움 - 부품 조달, 조립)이 높지만, 에너지와 연산 능력의 엄격한 한계로 인해 보안 위협에 더 취약함

# 2. 기존 딥러닝 방식의 한계

- **현황**: LSTM이나 Autoencoder(AE) 기반의 위성 전용 AD 모델들이 제안되어 왔음
- **치명적 약점**: 이러한 모델들은 높은 연산 부하와 그에 따른 **에너지 소모**를 수반함
- **현실적 문제**: 최고의 탐지 성능을 내는 고정 모델(Static Model)을 계속 돌리기엔 위성의 배터리가 버티지 못함

# 3. LighTellite의 핵심: 협력하는 이중 에이전트

- **동적 모델 선택(Dynamic Model Selection, DMS)**: 하나의 모델만 고집하지 않고, 상황에 맞춰 최적의 모델을 실시간 선택함
- **Dual-Agent 메커니즘**:
  - **에이전트 1 (Energy Manager)**: 사용 가능한 에너지 예산을 결정함
  - **에이전트 2 (Model Selector)**: 최신 데이터 분석을 통해 이상 탐지에 적합한 모델을 선별함

**Proposed Goal**

보안 성능과 에너지 효율 사이의 최적의 지점(Trade-off)을 찾아 자원 고갈 없는 지속 가능한 보안을 실현함

# 4. LighTellite의 주요 기여

- **최초의 에너지 인식형 AD**: 엄격한 에너지 제약 하에서 작동하는 위성용 AD 프레임워크 설계
- **RL 기반 DMS 적용**: 위성 도메인에 강화학습을 활용한 동적 모델 선택 기법을 최초로 도입함
- **정량적 우수성**: **AegisSat** 시뮬레이션 결과, 탐지율 10% 향상 및 에너지 21.8% 절감 달성
- **데이터 개방**: 58개 궤도의 큐브위성 시뮬레이션 데이터셋을 오픈소스로 공개함

# 03. Background

3.1 Satellites & CubeSats

# 1. 위성의 분류 및 궤도별 특성

- **역할**: 통신(Phone, TV), 내비게이션, 지구 관측 및 모니터링 서비스 제공

**궤도별 분류 (Orbital Categories)**

| 구분 | 고도 | 주요 특징 |
|---|---|---|
| **LEO** | 160 ~ 2,000km | 낮은 지연 시간, 고해상도 영상 (주기 90-120분) |
| **MEO** | 2,000 ~ 35,786km | 내비게이션(GPS) 및 통신 시스템 |
| **GEO** | 35,786km | 지구 자전 속도와 일치, 특정 지점 고정 |

- **환경적 공통점**: 제한된 자원, 지속적인 방사선 노출, 극심한 온도 변화라는 가혹한 환경에서 작동해야 함

# 2. 큐브위성의 물리적 사양 및 한계

- **정의**: 우주 접근 비용을 낮추기 위해 설계된 초소형 위성
- **표준 규격 (1U)**: $10 \times 10 \times 10$ cm 모듈 단위 구성
- **에너지 예산 (Energy Budget)**:
  - **평균 1 $\sim$ 2.5 Watts**의 전력만 사용 가능
  - 이는 태양광 패널의 좁은 표면적 때문에 발생하는 물리적 제약임

**응용 분야**

원격 탐사, 군사 감시, 교육 등 다양한 도메인에서 급격히 증가 중

# 3. 큐브위성의 보안 위협 요인

- **자원 착취형 공격**: 공격자는 위성의 한정된 에너지와 연산 능력을 노림
- **Denial-of-Service (DoS)**:
  - 자원 고갈(Exhaustion)이나 트래픽 과부하를 유발함
  - 온보드 에너지 예비 전력을 고갈시켜 임무 중단이나 완전한 고장을 초래함

**최대의 위험 구간: 일식 단계 (Eclipse Phase)**

태양광 충전이 불가능하고 오직 **배터리**로만 작동하는 시기
이때 DoS 공격이 발생하면 에너지가 빠르게 소모되어 위성이 회복 불가능한 상태가 될 수 있음

# Q1. LEO 주기 90-120분은 무엇을 의미하나요?

- **의미**: 저궤도(LEO) 위성이 **지구 한 바퀴를 완전히 도는 데 걸리는 시간**(공전 주기)을 말함
- **특징**:
  - 고도가 낮을수록 중력을 이기기 위해 더 빨리 돌아야 하므로 주기가 짧아짐
  - 위성은 하루에 약 12~16번 지구를 돌며 데이터를 수집하거나 통신을 수행함

**팁**

주기가 짧다는 것은 그만큼 지구 관측 해상도가 높지만, 통신 가시 시간(Window)이 짧아 관제소와의 빠른 데이터 교환이 필요하다는 뜻임.

# Q2. 큐브위성을 교육용으로도 사용하나요?

- **답변**: **네, 매우 활발하게 사용됩니다.**
- **이유**:
  - **저비용/표준화**: 수십억 원이 드는 대형 위성과 달리, 상대적으로 저렴한 비용으로 제작 가능함
  - **실습 위주 교육**: 대학생들이 직접 위성 설계, 제작, 발사, 운용의 전 과정을 경험할 수 있는 최고의 도구임
- **현황**: 전 세계 수많은 대학 연구실에서 우주 공학 및 보안 실습용으로 활용 중임

**Impact**

논문에서 사용된 데이터도 교육/연구용 테스트베드(AegisSat)에서 나온 실전 데이터입니다.

# Q3. 공격자가 자원을 고갈시킨다는 것이 무슨 뜻인가요?

- **개념**: 위성 내부 컴퓨터의 **연산 능력(CPU), 메모리, 저장공간**을 억지로 100% 사용하게 만드는 공격임
- **보안적 의미**:
  - 무의미한 연산을 반복하게 하거나 대량의 트래픽을 발생시킴
  - 컴퓨터가 바빠지면 **전력 소모가 급증**함

**Critical Impact**

전력이 한정된 위성에게 '연산 자원 고갈'은 단순한 시스템 지연이 아니라, **'배터리 방전을 통한 위성 사망'**을 유도하는 치명적인 공격임.

# Q4. 일식(Eclipse)은 달이 태양을 가리는 현상인가요?

- **교정**: 일반적인 일식(Solar Eclipse)과 달리, 위성 공학에서 말하는 일식 단계는 **'지구가 태양을 가리는 현상'**임
- **메커니즘**:
  - 위성이 궤도를 돌다가 **지구의 그림자 속**으로 들어가는 구간임
  - 이때 태양광 패널이 빛을 받지 못해 **태양광 발전이 중단**됨
- **위험성**: 오직 저장된 배터리로만 버텨야 하는 가장 취약한 시간임

**Summary**

이 시기에 공격자가 자원을 고갈시키면 위성은 '보조 전력'마저 잃고 영원히 잠들 수 (Dead) 있습니다. 그래서 LighTellite가 이 시기에 에너지를 아끼기 위한 매니저입니다.

# 03. Background

3.2 Reinforcement Learning (RL)

# 1. RL은 무엇이 다른가? (Paradigm Shift)

- **지도 학습(Supervised)**: 정답 라벨이 주어진 데이터를 학습함
- **비지도 학습(Unsupervised)**: 라벨 없는 데이터에서 패턴을 추출함
- **강화 학습(RL)**: 정답 대신 **보상(Reward)**을 통해 스스로 최적의 행동을 발견함

**학습 방식의 핵심 차이**

| 구분 | 피드백 | 학습 목표 |
|---|---|---|
| 지도 학습 | 정답($y$) | 정답과의 오차 최소화 |
| 강화 학습 | **보상($R$)** | **누적 보상의 최대화** |

**Trial and Error**

미리 정의된 정답이 없으며, 에이전트는 다양한 시도를 통해 보상을 최대화하는 행동을 찾아내야 함

# 1. 수학적 골격: MDP(Markov Decision Process) 5-Tuple

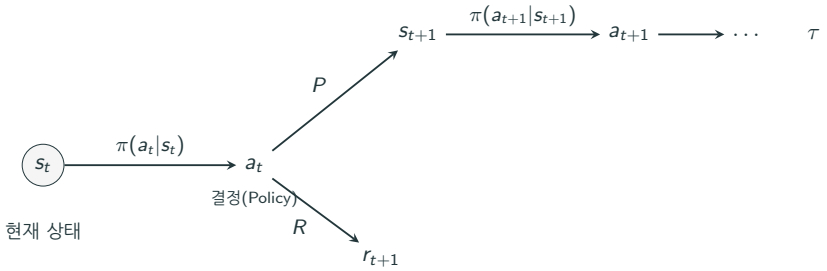
- MDP는 상태 전이와 보상이 **마르코프 성질(Markov Property)**을 따르는 확률 과정입니다.

**MDP 구성 요소의 수학적 정의**

| 기호 | 정의(Definition) | 수학적 의미 |
|---|---|---|
| $S$ | $s \in \mathcal{S}$ | 유한한 상태 공간 (Finite State Space) |
| $A$ | $a \in \mathcal{A}$ | 유한한 행동 공간 (Finite Action Space) |
| $P$ | $P(s'\|s,a) = \Pr(S_{t+1} = s'\|S_t = s, A_t = a)$ | **전이 확률**: 현재 $(s,a)$가 주어졌을 때 $s'$로 갈 확률 |
| $R$ | $R(s,a) = \mathbb{E}[R_{t+1}\|S_t = s, A_t = a]$ | **보상 함수**: 행동의 결과로 기대되는 즉각적인 보상 |
| $\gamma$ | $\gamma \in [0,1]$ | **할인 인자**: 미래 보상의 현재 가치 환산율 |

# 2. 에이전트-환경 상호작용 및 궤적($\tau$)

- 시간 $t$에 따라 상태($s$), 행동($a$), 보상($r$)이 사슬처럼 엮이며 **궤적(Trajectory)**을 만듭니다.




**궤적 (Trajectory)**

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots)$$

# 2-1. 정책(Policy, $\pi$): "어떻게 행동할 것인가?"

- **정의**: 특정 상태 $s$에서 어떤 행동 $a$를 취할지 결정하는 에이전트의 판단 전략
- **수학적 표현**: 확률 분포 $\pi(a|s) = \Pr(A_t = a|S_t = s)$

**위성 에이전트의 사례**

- **상태($s_t$)**: "현재 일식(Eclipse) 구간이며 배터리가 20% 남음"
- **행동($a_t$)**: "가벼운(Lightweight) VAE 모델을 선택하여 에너지 절약"
- **로직**: 위 상황에서 $a_t$를 선택할 확률을 계산하는 두뇌가 바로 $\pi$임

# 2-2. 궤적($\tau$)과 상태($s$): "무엇을 겪었는가?"

- **궤적(Trajectory, $\tau$)**: 임무 시작부터 끝까지 에이전트가 겪은 모든 상태, 행동, 보상의 전체 시퀀스

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$$

**상태(State): 시스템의 건전성 지표**

위성 보안에서 $s$는 단순한 물리적 위치가 아닌, 시스템의 **건전성(Health)**을 나타내는 벡터입니다.

- **전력 계통**: 배터리 전압, 전류 변화량, 남은 에너지 수준
- **컴퓨팅 자원**: CPU 점유율, 메모리 사용량, 시스템 인터럽트 횟수
- **임무 상황**: 현재 궤도 진행도(일식 여부), 최근 이상 탐지 이력

**Insight**

에이전트는 이 복잡한 상태 벡터($s$)를 보고, 위성의 생존과 보안을 모두 챙길 수 있는 최적의 정책($\pi$)을 수행해야 합니다.

# 3. 수학적 보상 합계: 수익(Return, $G_t$)

- 에이전트는 당장의 보상이 아닌, **미래까지의 총합**을 보고 행동합니다.

**감가 보상 합계 (Discounted Return)**

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- $\gamma = 0$: '근시안적' 에이전트. 당장의 보상($R_{t+1}$)만 고려함
- $\gamma \to 1$: '원시안적' 에이전트. 먼 미래의 보상까지 현재와 거의 동등하게 중요시함

# 4. 최적 정책($\pi^*$)의 수학적 설계

- 정책($\pi$)은 각 상태에서 행동을 취할 확률 분포 $\pi(a|s) = \Pr(A_t = a|S_t = s)$입니다.
- 우리의 목표는 **수익의 기댓값($\mathbb{E}$)**을 최대화하는 것입니다.

**RL Objective Function: $J(\pi)$**

$$J(\pi) = \mathbb{E}_{\tau \sim P_\pi}[G_0] = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t R_t \right]$$

- $\mathbb{E}_{\tau \sim P_\pi}$: 정책 $\pi$와 환경의 전이 확률 $P$가 만들어내는 모든 가능한 시나리오에 대한 평균
- **최종 미션**: $\pi^* = \arg\max_\pi J(\pi)$를 만족하는 지능형 정책을 찾는 것

# Section 3. Related Work

# 1. 위성 이상 탐지 기술의 분류

- 위성 AD 연구는 크게 세 가지 범주로 분류할 수 있습니다.

**AD 기술 카테고리 비교**

| 카테고리 | 주요 기술 | 특징 |
|---|---|---|
| **규칙 기반** | FDIR (결함 탐지, 격리 및 복구) | 전문가가 정의한 논리와 임계치 활용 |
| **정적 데이터 기반** | Clustering, SVM, Isolation Forest | 라벨 없는 데이터에서 편차 탐지 |
| **딥러닝 기반** | LSTM, Autoencoder (AE) | 시계열 및 다변량 패턴 학습 |

* FDIR(Fault Detection, Isolation, and Recovery)

**학습 포인트**

규칙 기반 및 정적 데이터 방식은 전문가 지식에 의존도가 높고, **새로운 형태의 이상 징후**에 대응하기 어렵다는 단점이 있음

# Q. 규칙 기반(Rule-based)은 일종의 정답지(라벨링)인가요?

- **답변**: **맞습니다. 전문가가 미리 정의한 "안전 가이드라인"입니다.**
- **작동 원리**: 전문가의 지식을 바탕으로 명확한 임계치(Threshold)나 IF-THEN 로직을 설정함
  - 예: "배터리 전압 $< 3.0V$ 이면 즉시 시스템 차단(FDIR)"
  - 예: "통신 패킷이 초당 100개 이상이면 DoS 공격으로 간주"
- **한계**:
  - 알려진 결함에는 강력하지만, **처음 보는 변칙적인 공격**에는 무방비함
  - 시스템이 복잡해질수록 모든 규칙을 사람이 직접 만드는 것이 불가능해짐

# Q. 정적 데이터 기반은 Clustering처럼 이상치를 분류하는 건가요?

- **답변**: 네, 데이터의 통계적/기하학적 분포를 이용해 **"혼자 떨어진 놈"을 찾는 것입니다.**
- **핵심 로직**: 라벨(정답)이 없어도 데이터끼리의 유사도를 계산하여 그룹화함
  - **Clustering**: 데이터 뭉치에서 멀리 떨어진 포인트를 이상치로 판단
  - **SVM / Isolation Forest**: 정상 데이터가 모여 있는 '영역'을 정의하고, 그 밖으로 나가는 데이터를 탐지
- **문제점**:
  - 데이터의 **시간적 흐름(Temporal pattern)**을 무시함
  - 위성 데이터는 "지금 이 값이 얼마인가"보다 "전후 맥락상 이 값이 정상인가"가 중요한데, 정적 모델은 이를 놓치기 쉬움

**Conclusion**

그래서 시간의 흐름을 읽는 **딥러닝(LSTM, VAE)**을 쓰는 것. 다만, 그만큼 계산량이 많아지니 LighTellite 같은 에너지 관리가 필수적임!

# 2. 위성 AD(Anomaly Detection)에서의 딥러닝 활용

- **등장 배경**: 위성 데이터는 복잡한 다변량 관계와 강한 시간적 의존성을 가짐
- **주요 접근법**:
  - **LSTM/RNN**: 정상 동작(Benign behavior)을 학습하고, 실제 측정값이 예측치와 크게 다를 때 (재구성 오차)를 이상으로 판단함
  - **AE + LSTM**: Autoencoder로 핵심 특징을 추출하고, LSTM으로 장기 의존성을 파악함
  - **임베디드 최적화**: 큐브위성의 마이크로컨트롤러에서 효율적으로 구동되도록 설계된 CNN/LSTM 모델 제안

# Q. 재구성 오차(Reconstruction Error)가 뭔가요?

- **답변**: 네, 본질적으로는 그렇습니다. 하지만 '정답' 없이 스스로 판단하는 방식입니다.
- **메커니즘 (Autoencoder 기준)**:
  1. **압축**: 입력 데이터 $x$를 아주 작은 정보로 요약함
  2. **복원**: 요약된 정보를 다시 원래 데이터 $\hat{x}$로 복구함
- **재구성 오차(Reconstruction Error)**: $e = \|x - \hat{x}\|^2$
  - **정상 데이터**: 평소 많이 봐서 복원(Reconstruct)을 아주 잘함 $\rightarrow$ $e$가 낮음
  - **이상 데이터**: 생전 처음 보는 패턴이라 복원을 못 하고 헤맴 $\rightarrow$ $e$가 높음

**비유하자면...**

고양이 사진만 본 AI에게 호랑이 사진을 주고 "그려봐"라고 하면, 고양이처럼 그리려다 실패. 그 '실패한 정도'가 바로 재구성 오차임.

# Q. 다변량(Multivariate)과 시간적 의존성이란 무엇인가요?

- **1. 다변량 (Multivariate)**: 여러 변수가 동시에 얽혀 있음
  - 수학적으로 현재 상태는 벡터 $\mathbf{x}_t = [v_1, v_2, \ldots, v_d]^T \in \mathbb{R}^d$임
  - 예: 전압($v$)이 낮아진 이유가 배터리 부족인지, 온도가 낮아서인지 여러 센서값을 동시에 봐야 함
- **2. 시간적 의존성 (Temporal Dependency)**: 과거가 현재에 영향을 미침
  - 수학적으로 현재값 $x_t$는 과거값들의 함수임: $x_t = f(x_{t-1}, x_{t-2}, \ldots)$
  - 예: 지금 배터리가 3.5V인 게 정상인지 보려면, 10분 전부터 서서히 떨어졌는지(정상) 갑자기 툭 떨어졌는지(이상)를 알아야 함

**Why LSTM/VAE?**

이 복잡한 관계($\mathbb{R}^d$)와 흐름($f$)을 사람이 규칙으로 다 적을 수 없기에, 딥러닝이 필요한 것임. (다변량 + 시간적 의존성)

# 3. 기존 정적(Static) 모델의 한계

- **고정된 모델의 문제**: 기존 연구들은 단일 고정 모델에 의존하며, 데이터 분포의 변화나 온보드 자원 제약에 **동적으로 대응하지 못함**

**정적 AD 모델의 트레이드오프**

| 모델 유형 | 특징 및 한계 |
|---|---|
| **고성능 모델** | 높은 탐지 정확도를 확보하지만, 에너지를 과도하게 소모함 |
| **저전력 모델** | 에너지 효율은 높으나 탐지 정확도가 떨어짐 |

**LighTellite의 차별점**

상황에 따라 모델을 교체하는 **적응성(Adaptability)**을 통해, 낮은 에너지 소모량을 유지하면서도 높은 탐지 신뢰성을 확보함

# Section 4. Methodology

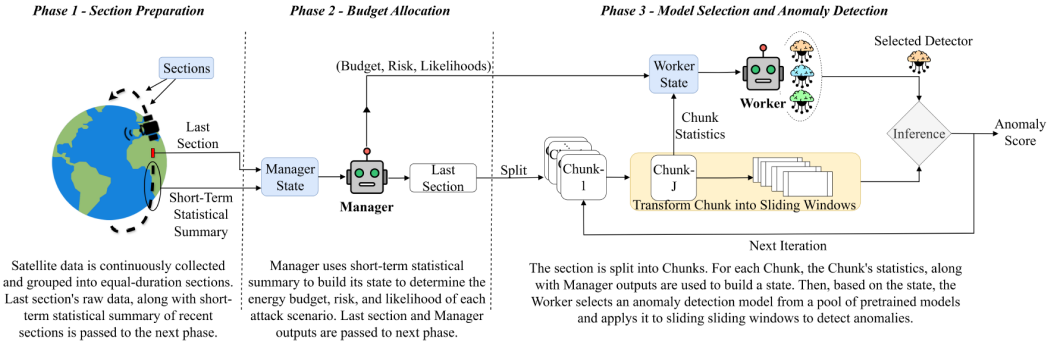
# Methodology Overview: LighTellite Pipeline

- **목표**: 온보드 데이터 패턴과 자원 상태에 따른 AD 성능 및 에너지 효율의 동적 균형
- **시스템 구조**: 총 3단계(Phase)의 순차적 파이프라인으로 구성됨

**핵심 설계 철학**

거시적 결정(Manager)과 미시적 실행(Worker)의 분리를 통한 효율적 자원 관리

# Methodology Overview: LighTellite Pipeline




LighTellite Framework의 전체 워크플로우

# Phase 1. Section Preparation: 데이터 그룹화

- **수집 단계**: 위성 궤도 운용 중 실시간 센서 데이터(Sensor Readings)를 지속적으로 읽어들임
- **데이터 단위**: 수집된 데이터를 동일한 시간 길이의 **'섹션(Section)'** 단위로 그룹화함
- **추출 정보**:
  - Raw 데이터 (최근 섹션의 가공되지 않은 센서값)
  - 단기 통계 요약(Short-term Statistical Summary): 평균, 분산 등 통계적 특징

**Input for Next Phase**

이 섹션 정보와 통계 요약본이 Manager 에이전트의 판단 근거(State)가 됩니다.

# Q. 위성은 대체 어떤 Raw 데이터를 수집하고 있나요?

- **질문 요약**: LighTellite가 처리하는 '섹션' 내 Raw 데이터에는 어떤 구체적인 센서값들이 포함되어 있나요?
- **답변**: 큐브위성의 상태를 나타내는 **텔레메트리(Telemetry)** 데이터로, 크게 3가지 카테고리로 나뉩니다.

## AegisSat 기반 주요 수집 항목

| 카테고리 | 구체적 데이터 항목 (Raw Features) |
|---|---|
| **전력 계통 (EPS)** | 버스 전압(Voltage), 배터리 전류(Current), 태양광 패널 발전량, 전력 소모량 |
| **컴퓨팅 자원 (OBC)** | CPU 사용률(%), 메모리 점유율, 입출력(I/O) 작업 횟수, 프로세스 실행 시간 |
| **시스템 환경** | 내부/외부 온도, 통신 패킷 수(RF Traffic), 태양 가시성(일식 여부 플래그) |

## 공학적 포인트

이 데이터들은 일정한 샘플링 주기에 따라 수집되어, 하나의 시점($t$)에서 수십 개의 차원을 가진 **다변량 벡터($\mathbf{x}_t$)**를 구성하게 된단다.

# Q. Raw 데이터가 이상 탐지에 미치는 영향

- **데이터의 결합**: 단일 센서값보다는 여러 값의 **상관관계**가 중요함
  - 예: CPU 사용량이 급증했는데 전압이 떨어진다? → 공격에 의한 자원 고갈 의심
  - 예: 일식 구간이 아닌데 발전량이 0이다? → 하드웨어 결함 의심
- **통계적 요약과 Raw Data의 역할**:
  - **Worker - Raw Data**: 미세한 떨림이나 즉각적인 이상 징후 포착
  - **Manager - Summary Data**: 평균, 분산 등을 통해 에너지 소모 트렌드나 시스템의 전반적인 건강 상태 파악

**Note**

결국 이 수많은 센서 데이터가 에이전트에게는 '눈'과 같다. 이 눈을 통해 현재 위성이 안전한지, 전기가 모자란지를 판단함.

# Phase 2. Budget Allocation: 거시적 자원 배분

- **주체**: **Manager 에이전트** (지휘관 역할)
- **상태 구성(Manager State)**: 입력된 섹션 데이터를 바탕으로 현재 위성의 자원 상황을 파악함
- **결정 사항 (Manager Outputs)**:
  1. **에너지 예산(Energy Budget)**: 이번 섹션에서 사용할 총 에너지양 결정
  2. **위험 지수(Risk Factor)**: 현재 비정상 활동이 발생할 확률 계산
  3. **공격 시나리오 가능성**: 각 공격 타입별 발생 확률 예측

**Role of Manager**

"지금은 위험하니 전기를 좀 더 써서 꼼꼼히 보자" 혹은 "평온하니 에너지를 아끼자"와 같은 전략적 판단을 내립니다.

# Phase 3. Model Selection & AD: 미시적 탐지 실행

- **주체**: **Worker 에이전트** (행동대장 역할)
- **세부 공정**:
  1. 섹션을 더 작은 단위인 **'청크(Chunk)'**로 나눔
  2. **Contextual State 구축**: 전압/전류 변동, 에너지 트렌드, 최근 이상 이력을 결합
  3. **동적 모델 선택(Dynamic Model Selection, DMS)**: 사전 학습된 모델 풀(Pool)에서 최적의 모델 하나를 선택함
- **탐지(Inference)**: 선택된 모델을 슬라이딩 윈도우 방식으로 적용하여 **이상 점수(Anomaly Score)** 산출

**The Core of DMS**

각 모델은 성능과 에너지 소모량이 다르기 때문에, 자원 제약 내에서 가장 '가성비' 좋은 모델을 실시간으로 골라내는 것이 핵심입니다.

# Phase 1 - Section Preparation

# 1. 궤도의 수학적 정의: Orbit and Sections

- **정의**: 하나의 궤도($O_j$)는 동일한 시간 길이를 가진 유한한 세그먼트인 **'섹션(Section)'** 들의 시퀀스로 모델링됩니다.

**수학적 표현 (Equation)**

$$O_j = [S_{j,1}, S_{j,2}, \ldots, S_{j,n}]$$

- $O_j$: $j$번째 궤도 전체
- $S_{j,i}$: $j$번째 궤도 내의 $i$번째 섹션 데이터

- **공학적 의미**: 위성이 지구를 한 바퀴 도는 동안, LighTellite는 $n$번의 의사결정 구간을 나누어 주기적으로 보안 상태를 점검함

# 2. 다변량 데이터의 구조화: Feature Groups

- **개념**: 각 섹션 내의 수많은 센서 데이터를 성격에 따라 $K$개의 그룹으로 분류합니다.
- **수학적 정의**: $F = \{F_1, F_2, \ldots, F_K\}$

**Feature Grouping 사례**

| 그룹 ($F_k$) | 모니터링 대상 | 해당 센서 예시 |
|---|---|---|
| $F_{telemetry}$ | 전력 시스템 (EPS) | 전압, 전류, 발전량, 소모량 |
| $F_{system}$ | 컴퓨팅 자원 (OBC) | CPU 점유율, 메모리, 네트워크 활동 |

**Grouping의 목적**

특정 리소스(예: 전력)의 이상 징후가 다른 리소스(예: 연산)에 미치는 영향을 독립적이면서도 유기적으로 분석하기 위함임

# 3. 입력 데이터 합성: Raw + Summary

- 매 활성화 주기마다 Manager 에이전트는 두 종류의 데이터를 입력받습니다.

1. **최신 Raw 데이터 ($S_{j,i}$)**:
   - 현재 섹션에서 수집된 가장 따끈따끈한 원시 센서값
   - **역할**: 즉각적이고 미세한 변화(Spike) 탐지
2. **단기 통계 요약 (Statistical Summary)**:
   - 현재 섹션 이전($S_{j,i-1}, S_{j,i-2}, \ldots$)의 여러 섹션에서 유도된 통계치
   - **역할**: 시스템의 에너지 소모 추세 및 전반적인 건강 상태 파악

**Next Step: Phase 2**

이 합성된 정보를 기반으로 Manager는 "이번 섹션에서 전기를 얼마나 쓸 것인가?"를 결정하게 됩니다.

# Phase 2 - Budget and Risk Assessment

# 1. Manager의 입력 정보: 상황 파악

- **Input**: 최신 섹션의 Raw 데이터($S_{j,i}$) + 이전 섹션들의 통계적 요약(Summary)
- **상태 벡터($s_i$) 구성**: 위성의 운영 문맥(Operational Context)을 캡슐화함

**State Vector ($s_i$)의 주요 요소**

- **에너지 상태**: 정규화된 배터리 잔량, 에너지 소모 트렌드
- **임무 진행도**: 궤도 진행률(Orbit Progress)
- **시스템 건전성**: 섹션 단위의 텔레메트리 및 시스템 통계
- **보안 이력**: 최근 발생한 이상 탐지 기록

**Note**

지휘관은 단순히 현재 전기가 얼마 남았는지만 보는 게 아니라, "최근에 이상 징후가 나타난 적이 있는가?"까지 고려해서 예산을 짭니다.

# Q. 운영 문맥을 캡슐화한다는 게 구체적으로 무슨 뜻인가요?

- **질문 요약**: 최신 데이터와 과거 통계치를 합쳐서 '문맥'을 만든다는 것이 맞나요?
- **답변**: 네, 맞습니다. "현재 어디서 무엇을 하고 있는가"를 에이전트가 이해하도록 데이터를 조합하는 과정입니다.

**입력 데이터와 운영 문맥의 매핑 (Mapping)**

| 입력 소스 (Inputs) | 생성되는 운영 문맥 (Operational Context) |
|---|---|
| **최신 Raw ($S_{j,i}$)** | 현재 배터리 전압, CPU 점유율 등 **'지금 이 순간'**의 상태 |
| **통계 요약 (Summary)** | 최근 1시간 동안의 에너지 소모 **'트렌드'** 및 평균 부하 |
| **궤도 정보 ($i/n$)** | 현재 위성이 태양광을 받는지(발전), 그림자인지(일식)의 **'진행도'** |
| **보안 이력** | 방금 전 청크에서 이상 점수가 높았는지에 대한 **'과거 기억'** |

# Q. 왜 '단일 시점' 데이터만으로는 부족할까요?

- **사례: 전압이 3.2V로 떨어졌을 때**
  - **문맥 없는 에이전트**: "어? 전압이 낮네? 공격인가? (당황)"
  - **문맥을 아는 에이전트**: "아, 지금은 일식 구간(궤도 진행도)이고, 최근 통계 요약을 보니 서서히 떨어지는 중이야. 이건 공격이 아니라 정상적인 소모야. 안심하자."

**Note**

데이터를 캡슐화(Encapsulation)한다는 것은, 흩어진 센서값들을 모아 에이전트가 즉각적으로 판단할 수 있는 하나의 '의미 있는 상태 벡터($s_i$)'로 만드는 과정임.

# 2. 보상 함수의 재료: 무엇을 잘해야 상을 주는가?

- Manager의 정책 학습을 가이드하는 4가지 핵심 지표입니다.

**Reward Components**

| 지표 | 수식 | 공학적 의미 |
|---|---|---|
| **탐지 점수** | $R_{det} = F + P$ | Worker의 F1 스코어($F$)와 임무 완수율($P$)의 합 |
| **에너지 패널티** | $E_i$ | 이번 섹션에서 Worker가 실제로 쓴 추론 에너지 |
| **예산 오차** | $B_{ex}$ | 할당한 예산($B_i$)과 실제 소모량 사이의 간극 (과잉 할당 방지) |
| **고갈 패널티** | $D$ | 배터리 방전 시 남은 궤도 비율 (조기 종료 방지) |

# Q. F1 스코어($F$)와 임무 완수율($P$)은 무엇인가요?

- **F1 스코어 ($F$)**: 정밀도(Precision)와 재현율(Recall)의 조화 평균
  - **의미**: "공격을 놓치지 않으면서도(Recall), 가짜 알람을 울리지 않는(Precision)" 균형 잡힌 성능 지표
- **임무 완수율 ($P$)**: 이번 섹션에서 처리해야 할 전체 청크 중 **실제로 처리한 청크의 비율**
  - **수식**: $P = \frac{\text{완료된 청크 수}}{\text{섹션 내 총 청크 수}}$
  - **사례**: 에너지가 부족해 10개 청크 중 8개만 검사하고 멈췄다면 $P = 0.8$

**Note**

탐지 성능($F$)이 아무리 좋아도, 전기가 모자라 검사를 끝까지 못 하면($P < 1$) 보안에 구멍이 뚫린 것이나 다름없다. 그래서 두 값을 합쳐서 평가하는 것임.

# Q. 예산 오차($B_{ex}$)와 패널티($D$)의 수치적 의미

- **예산 오차 ($B_{ex}$)**: 관리자의 할당 예산($B_i$)과 실제 소모량($E_i$)의 차이
  - **수식**: $B_{ex} = \max(0, B_i - E_i)$
  - **의도**: 관리자가 에너지를 너무 '펑펑' 허락하지 않도록 조절함 (자원 최적화)
- **고갈 패널티 ($D$)**: 배터리 방전 시 **남은 궤도(섹션)의 비율**
  - **예시**: 총 10개 섹션 중 8번째에서 배터리가 다 나갔다면, 남은 2개 섹션에 대한 벌점 $D = 2/10 = 0.2$
  - **의도**: 에이전트가 단기 탐지 성능에 눈이 멀어 배터리를 일찍 다 써버리는 '조기 탈락'을 방지함

**핵심 요약**

$B_{ex}$는 "낭비하지 마라", $D$는 "끝까지 살아남아라"라는 교훈을 에이전트에게 주는 수단임.

# 3. 수학적 최적화: 통합 보상 함수 $R_i$

- Manager는 다음의 가중치 합을 최대화하도록 학습됩니다.

**The Reward Equation**

$$R_i = w_1 Y_i R_{det,i} - w_2 E_i - w_3 B_{ex,i} - (1 - Y_i) w_4 L_i - w_5 D_i$$

- $Y_i \in \{0, 1\}$: 현재 섹션에 공격이 존재하는지 여부
- $w_1, \ldots, w_5$: 각 목표의 중요도를 조절하는 가중치(Weights)
- $L_i$: 정상 상황임에도 Manager가 공격 가능성을 높게 판단했을 때의 패널티

**Key Logic**

공격이 있을 때($Y_i = 1$)는 탐지 성능($R_{det}$)이 중요하고, 평시($Y_i = 0$)에는 불필요한 과잉 탐지($L_i$)를 줄여야 한단다!

# 4. Phase 2 Output: Worker에게 내리는 명령

- Manager가 모든 판단을 마치면 다음 정보를 다음 단계로 넘깁니다.

1. **할당된 예산 ($B_i$)**: 이번 섹션에서 쓸 수 있는 최대 에너지 주머니
2. **위험 요인 (Risk Factor)**: 현재 환경의 위험 수준
3. **공격 시나리오 가능성 (Likelihoods)**: 어떤 종류의 공격이 예상되는가?
4. **섹션 데이터 ($S_{j,i}$)**: 실제 분석할 Raw 데이터

**Transition to Phase 3**

이제 행동대장(Worker)은 이 예산과 위험 정보를 들고, 실전에서 어떤 모델을 돌릴지 결정하게 됩니다.

# 4-1. 매니저의 Raw 데이터 + 단기 요약 데이터에서 결정으로: Mapping Process

- Manager 내부에는 데이터를 판단 결과로 변환하는 **정책 신경망($\pi_\theta$)**이 존재합니다.

1. **상태 구성 (State Construction)**:
   - 섹션 Raw($S_{j,i}$)와 이전 통계를 하나의 긴 벡터($s_i$)로 결합(Concatenation)함
   - 이때 모든 값은 신경망이 잘 이해하도록 $0 \sim 1$ 사이로 **정규화(Normalization)**됨
2. **신경망 통과 (Neural Inference)**:
   - 결합된 벡터 $s_i$가 심층 신경망(DNN)의 입력층으로 들어감
   - 다중 레이어를 거치며 데이터 간의 복잡한 상관관계를 추론함

# 4-2. 출력값의 결정: Budget, Risk, Likelihoods

- 신경망의 마지막 층(Output Layer)은 우리가 원하는 3가지 값을 내놓습니다.

**Manager의 출력값(Action) 도출**

| 출력 항목 | 활성화 함수 | 도출 의미 |
|---|---|---|
| **예산 ($B_i$)** | Sigmoid/Linear | 이번 섹션에서 허용할 에너지의 총량 (Joule 단위) |
| **위험도 (Risk)** | Sigmoid ($0 \sim 1$) | 현재 환경이 비정상(Anomaly)일 확률 |
| **공격 가능성** | Softmax | DoS, 자원 고갈 등 각 시나리오가 발생할 상대적 확률 |

**How does it know?**

이 신경망은 지난 슬라이드에서 배운 **보상 함수($R_i$)**를 최대화하는 방향으로 수만 번의 시행착오를 거치며 '예산 짜는 법'을 스스로 터득한 것입니다.

# 4-3. Manager의 신경망 업데이트: 전략적 진화

- **주체**: Manager Policy Network ($\pi_{\theta_M}$)
- **학습 목표**: 보상 함수 $R_i$의 기댓값 최대화

**업데이트 로직 (The Learning Loop)**

1. **Action**: 예산($B_i$)과 위험도를 결정함
2. **Feedback**: Worker가 실제 수행한 결과(탐지 점수, 에너지 소모)를 관찰함
3. **Reward**: $R_i$를 계산(일종의 '칭찬' 혹은 '벌' 개념)
4. **Update**: 보상이 낮으면 신경망 가중치($\theta_M$)를 수정하여 다음번에 더 나은 예산을 짜도록 함

# 4-Final. Worker의 신경망 업데이트: 모델 선택의 묘미

- **주체**: Worker Policy Network ($\pi_{\theta_W}$)
- **학습 목표**: Manager가 준 예산($B_i$) 내에서 탐지 성능 최대화

**Worker의 Action: Dynamic Model Selection**

| 상태(State) | 행동(Action: 모델 선택) |
|---|---|
| 예산이 넉넉하고 위험함 | **Heavy VAE**: 에너지를 써서라도 확실히 탐지 |
| 예산이 적고 평온함 | **Light VAE**: 에너지를 아끼며 최소한의 감시 유지 |

**공학적 결론**

두 에이전트는 서로 독립된 신경망을 가지지만, **'예산'**이라는 고리로 묶여 함께 성장합니다. 이걸 **계층적 강화학습(HRL)** 구조라고 부름.

# Q. 계층적 강화학습(HRL)이란

- 단순히 "누가 더 높은가"의 문제가 아니라, **추상화 수준**의 차이입니다.

1. **시간적 계층 (Temporal Hierarchy)**:
   - **Manager**: 거시적 시간(Section 단위)으로 세상을 바라봄
   - **Worker**: 미시적 시간(Chunk 단위)으로 세상을 바라봄
2. **목표의 계층 (Goal Hierarchy)**:
   - **Manager**: 전체 임무의 '생존'과 '에너지 분배'라는 **상위 목표**를 설정
   - **Worker**: 주어진 예산 내에서 '정확한 탐지'라는 **하위 목표**를 수행

# Q. 왜 복잡하게 계층을 나누나요?

- 모든 것을 하나의 에이전트가 처리하면 다음과 같은 문제가 발생합니다.

**단일 구조 vs 계층적 구조**

| 구분 | 단일 에이전트 (Flat) | 계층적 에이전트 (HRL) |
|---|---|---|
| **학습 난이도** | 너무 많은 변수를 한꺼번에 배워야 해서 학습이 매우 느림 | 문제를 전략과 전술로 나누어 효율적으로 학습 가능 |
| **유연성** | 환경이 조금만 바뀌어도 전체를 다시 학습해야 함 | 예산 정책(Manager)이나 모델 선택(Worker) 중 하나만 수정 가능 |

**Professor's Note**

마치 총장이 대학의 큰 예산을 배정하면, 학과장이 그 안에서 필요한 실습 도구(모델)를 사는 것과 같음. 이렇게 역할을 나누어야 위성처럼 복잡한 시스템을 효율적으로 관리할 수 있지!

# 질의 & 응답

Any Question?